

Data Visualisation & Analytics

Endterm Examination Notes

Module I: Google Sheets **Module II:** Python **Module III:** Tableau

Comprehensive revision notes covering the complete syllabus

Contents

Quick Reference Cheat Sheet	3
1 Introduction to DVA	4
1.1 Data Analysis & Visualisation	4
1.2 Types of Analytics	4
1.3 Analytics Lifecycle	4
2 Google Sheets	5
2.1 Interface Overview	5
2.2 Cell Referencing	5
2.3 Data Types	5
2.4 Sorting and Filtering	5
2.5 Data Cleaning & Standardisation	5
2.5.1 Handling Missing Values	5
2.5.2 Find & Replace	6
2.5.3 Text to Columns / Split	6
2.5.4 Useful Cleaning Functions	6
2.6 Functions	6
2.6.1 Number Functions	6
2.6.2 String / Text Functions	6
2.6.3 Date & Time Functions	7
2.6.4 Logical Functions	7
2.6.5 Aggregate Functions	7
2.7 Conditional Aggregations	8
2.8 Conditional Formatting	8
2.9 Lookup Functions	8
2.9.1 VLOOKUP	9
2.9.2 HLOOKUP	9
2.9.3 XLOOKUP (Modern Alternative)	9
2.10 INDEX-MATCH	9
2.11 Error Understanding & Handling	10
2.12 Pivot Tables & Charts	10
2.12.1 Pivot Tables	10
2.12.2 Charts	10
2.13 Array Formulas	11
2.14 What-If Analysis	11
2.14.1 Goal Seek	11
2.14.2 Scenario Manager	11
2.15 Dashboard & Report Creation	11
3 Python for Data Analysis	12
3.1 Key Libraries Overview	12
3.2 NumPy	12
3.2.1 NumPy Arrays	12
3.2.2 Indexing & Slicing	12
3.2.3 Boolean Masking	12
3.2.4 Vector, Numerical & Matrix Operations	13
3.3 Pandas	13
3.3.1 Data Loading	13

3.3.2	Data Analysing & Cleaning	14
3.3.3	Data Filtering	14
3.3.4	Data Grouping	14
3.3.5	Data Merging	15
3.3.6	Data Pivoting	15
3.4	Descriptive Statistics	15
3.5	Frequency Analysis	16
3.6	Correlation Analysis	16
3.7	Outlier Detection	16
3.7.1	IQR Method	16
3.7.2	Z-Score Method	17
3.8	Window Functions	17
3.9	Data Distributions	17
3.10	Univariate, Bivariate & Multivariate Analysis	17
3.10.1	Univariate — One Variable	17
3.10.2	Bivariate — Two Variables	18
3.10.3	Multivariate — Three or More Variables	18
4	Tableau	19
4.1	Interface	19
4.2	Data Types	19
4.3	Data Connections	19
4.4	Dimensions & Measures	19
4.5	Discrete vs Continuous (Green vs Blue)	20
4.6	Marks Card & Charts	20
4.7	Filters & Their Types	20
4.8	Relationships, Joins, Union, Data Blending	20
4.9	Calculated Fields & Table Calculations	21
4.9.1	Calculated Fields	21
4.9.2	Table Calculations	21
4.10	Level of Detail (LOD) Expressions	21
4.10.1	FIXED LOD	22
4.10.2	INCLUDE LOD	22
4.10.3	EXCLUDE LOD	22
4.11	Sets & Parameters	22
4.11.1	Sets	22
4.11.2	Parameters	22
4.12	Groups, Bins, Hierarchies	23
4.12.1	Groups	23
4.12.2	Bins	23
4.12.3	Hierarchies	23
4.13	Containers, Tiles (Layout Containers)	23
4.14	Dashboard Design & Storyboard	23
4.14.1	Dashboard Design	23
4.14.2	Story (Storyboard)	24
	Exam Preparation Summary	25

Quick Reference Cheat Sheet

Key Formulas at a Glance

Topic	Quick Syntax
VLOOKUP	=VLOOKUP(key, range, col_index, 0)
XLOOKUP	=XLOOKUP(key, lookup_arr, return_arr)
INDEX-MATCH	=INDEX(return_range, MATCH(key, lookup_range, 0))
COUNTIFS	=COUNTIFS(range1, crit1, range2, crit2)
SUMIFS	=SUMIFS(sum_range, range1, crit1)
Pandas groupby	df.groupby('col').agg({'col2':'sum'})
Pandas merge	pd.merge(df1, df2, on='key', how='inner')
LOD Fixed	{FIXED [Dim] : SUM([Measure])}
LOD Include	{INCLUDE [Dim] : AVG([Measure])}
LOD Exclude	{EXCLUDE [Dim] : COUNT([Measure])}

1 Introduction to DVA

1.1 Data Analysis & Visualisation

Definition

Data Analysis is the process of inspecting, cleansing, transforming, and modelling data to discover useful information, draw conclusions, and support decision-making.

Data Visualisation is the graphical representation of data to communicate insights clearly and efficiently.

1.2 Types of Analytics

Type	Question Answered	Example
Descriptive	What happened?	Monthly sales report
Diagnostic	Why did it happen?	Drop in revenue analysis
Predictive	What will happen?	Forecasting next quarter sales
Prescriptive	What should we do?	Recommending optimal pricing

Exam Tip

Descriptive → Diagnostic → Predictive → Prescriptive is a progression from **hindsight** to **foresight** to **action**.

1.3 Analytics Lifecycle

1. **Problem Definition** — Identify the business question or goal.
2. **Data Collection** — Gather raw data from relevant sources.
3. **Data Cleaning** — Handle missing values, duplicates, inconsistencies.
4. **Exploratory Data Analysis (EDA)** — Understand distributions, patterns, outliers.
5. **Data Modelling / Analysis** — Apply statistical or ML methods.
6. **Visualisation** — Present findings using charts and dashboards.
7. **Reporting & Decision Making** — Communicate insights to stakeholders.

2 Google Sheets

2.1 Interface Overview

- **Workbook:** The entire Sheets file; contains multiple *worksheets*.
- **Worksheet / Sheet:** A single tab (grid of rows and columns).
- **Cell:** Intersection of a row and column (e.g., B3).
- **Formula Bar:** Shows/edits the content of the active cell.
- **Name Box:** Displays the active cell reference; can jump to any cell.

2.2 Cell Referencing

Types of Cell References

Type	Syntax	Behaviour on Copy
Relative	A1	Both row and column adjust
Absolute	\$A\$1	Row and column are locked
Mixed (col fixed)	\$A1	Column locked; row adjusts
Mixed (row fixed)	A\$1	Row locked; column adjusts

\$ locks the letter or number immediately following it. Press **F4** to cycle through reference types.

2.3 Data Types

- **Number:** Integers, decimals; right-aligned by default.
- **Text / String:** Alphanumeric characters; left-aligned.
- **Date & Time:** Stored as serial numbers; formatted for display.
- **Boolean:** TRUE / FALSE.
- **Error:** #N/A, #VALUE!, #REF!, etc.

2.4 Sorting and Filtering

- **Sort:** Data → Sort range. Choose column and ascending/descending.
- **Multi-level Sort:** Add multiple sort columns with priority.
- **Filter:** Data → Create a filter. Click dropdown arrows on headers to filter by value, condition, or colour.
- **SORT function:** =SORT(range, sort_col, 1/-1, [by_col])
- **FILTER function:** =FILTER(range, condition1 * condition2) — returns dynamic array.

2.5 Data Cleaning & Standardisation

2.5.1 Handling Missing Values

- **Delete rows:** Manually remove or filter blank rows.

- **Fill with value:** Use `=IF(ISBLANK(A1), "Unknown", A1)`.
- **Fill with average:** `=IFERROR(A1, AVERAGE(A:A))`.
- **ISBLANK(cell):** Returns TRUE if cell is empty.

2.5.2 Find & Replace

- **Shortcut:** Ctrl+H.
- Can match case, match entire cell contents, or use regex.

2.5.3 Text to Columns / Split

- **SPLIT:** `=SPLIT(text, delimiter)` — splits into multiple columns.
- **Data → Split text to columns:** GUI alternative.

2.5.4 Useful Cleaning Functions

- **TRIM(text)** — removes extra spaces.
- **CLEAN(text)** — removes non-printable characters.
- **UPPER/LOWER/PROPER(text)** — standardises case.
- **SUBSTITUTE(text, old, new)** — replaces specific strings.
- **IFERROR(formula, val)** — returns *val* if formula errors.

2.6 Functions

2.6.1 Number Functions

Function	Description / Syntax
<code>ROUND(n, d)</code>	Round to <i>d</i> decimal places
<code>ROUNDUP / ROUNDDOWN</code>	Always round up or down
<code>INT(n)</code>	Floor to integer
<code>MOD(n, div)</code>	Remainder of division
<code>ABS(n)</code>	Absolute value
<code>POWER(base, exp)</code>	Exponentiation
<code>SQRT(n)</code>	Square root
<code>CEILING / FLOOR(n, f)</code>	Round to nearest multiple of <i>f</i>
<code>RAND()</code>	Random decimal 0–1
<code>RANDBETWEEN(a, b)</code>	Random integer in range

2.6.2 String / Text Functions

Function	Description
LEN(text)	Number of characters
LEFT(text, n)	First <i>n</i> characters
RIGHT(text, n)	Last <i>n</i> characters
MID(text, start, n)	<i>n</i> chars starting at <i>start</i>
FIND(search, text)	Position of search string (case-sensitive)
SEARCH(search, text)	Position (case-insensitive)
CONCATENATE / &	Join strings: =A1&" "&B1
TEXT(val, fmt)	Format number as text: TEXT(A1,"dd/mm/yyyy")
VALUE(text)	Convert text to number
REPT(text, n)	Repeat text <i>n</i> times

2.6.3 Date & Time Functions

Function	Description
TODAY()	Current date (dynamic)
NOW()	Current date + time
DATE(y, m, d)	Create a date value
DAY / MONTH / YEAR(date)	Extract part of a date
WEEKDAY(date, 2)	Day of week (1=Mon with type 2)
WEEKNUM(date)	Week number in year
DATEDIF(start, end, unit)	Difference in "D", "M", "Y"
EDATE(date, months)	Date <i>n</i> months away
EOMONTH(date, months)	Last day of month <i>n</i> months away
NETWORKDAYS(s, e)	Working days between two dates

2.6.4 Logical Functions

- IF(cond, val_true, val_false) — basic conditional.
- IFS(cond1, val1, cond2, val2, ...) — multiple conditions.
- AND(c1, c2, ...) — TRUE only if all conditions are TRUE.
- OR(c1, c2, ...) — TRUE if any condition is TRUE.
- NOT(cond) — reverses TRUE/FALSE.
- SWITCH(expr, val1, res1, val2, res2, ..., default) — match-based.

2.6.5 Aggregate Functions

SUM(range)	Total of all numbers
AVERAGE(range)	Arithmetic mean
MEDIAN(range)	Middle value
MODE(range)	Most frequent value
COUNT(range)	Count of numbers
COUNTA(range)	Count of non-empty cells
COUNTBLANK(range)	Count of empty cells
MAX / MIN(range)	Largest / smallest value
LARGE(range, k)	k -th largest value
SMALL(range, k)	k -th smallest value
STDEV / VAR(range)	Standard deviation / variance (sample)

2.7 Conditional Aggregations

```
=COUNTIF(range, criterion)
=COUNTIFS(range1, crit1, range2, crit2, ...)
Example: Count orders > 100: =COUNTIF(B:B, ">100")
```

```
=SUMIF(range, criterion, sum_range)
=SUMIFS(sum_range, range1, crit1, range2, crit2, ...)
Example: Sum sales for "North" region: =SUMIF(A:A,"North",C:C)
```

```
=AVERAGEIF(range, criterion, average_range)
=AVERAGEIFS(avg_range, range1, crit1, ...)
```

Key Difference

In SUMIF: order is (criteria range, criterion, sum range). In SUMIFS: sum range comes **first**.

2.8 Conditional Formatting

- **Access:** Format → Conditional Formatting.
- Apply colour rules based on: *cell value, text contains, date is, custom formula*.
- **Colour Scales:** Gradient from low to high values (great for heat maps).
- **Custom Formula** example — highlight duplicates: `=COUNTIF($A:$A, A1)>1`
- Rules are evaluated in order; first match wins.

2.9 Lookup Functions

2.9.1 VLOOKUP

```
=VLOOKUP(search_key, range, index, [is_sorted])
```

- **search_key**: Value to look up.
- **range**: Table to search (key must be in **first column**).
- **index**: Column number to return (1 = first column of range).
- **is_sorted**: FALSE for exact match (use almost always).

VLOOKUP Limitations

- Can only look **left-to-right** — key must be the first column.
- Returns value from the **first match** only.
- Column index is fragile — breaks when columns are inserted.

2.9.2 HLOOKUP

```
=HLOOKUP(search_key, range, row_index, [is_sorted])
```

Same as VLOOKUP but searches across **rows** instead of columns.

2.9.3 XLOOKUP (Modern Alternative)

```
=XLOOKUP(search_key, lookup_array, return_array, [if_not_found],  
[match_mode], [search_mode])
```

- Works in any direction (left, right, up, down).
- Returns entire row/column if return_array spans multiple columns.
- **match_mode**: 0 = exact, -1 = exact or next smaller, 1 = next larger.

It is more flexible, readable, and does not break when columns are inserted.

2.10 INDEX-MATCH

```
=INDEX(return_range, MATCH(search_key, lookup_range, 0))
```

- **MATCH** finds the *row position* of the key in the lookup range.
- **INDEX** returns the value at that position in the return range.
- Can look in any direction, and the return column can be to the **left** of the lookup column.
- 0 as the third argument of MATCH = exact match.

MATCH type argument

1 = less than (sorted ascending) 0 = exact match -1 = greater than (sorted descending)

2.11 Error Understanding & Handling

Error	Cause	Fix
#N/A	Lookup value not found	Use IFERROR or IFNA
#VALUE!	Wrong data type	Check argument types
#REF!	Invalid cell reference	Restore deleted cells/range
#DIV/0!	Division by zero	Wrap with IF(denom=0,...)
#NAME?	Misspelled function name	Correct spelling
#NUM!	Invalid numeric value	Check inputs (e.g., SQRT(-1))
#NULL!	Incorrect range operator	Use comma or colon correctly

=IFERROR(formula, value_if_error) — catches any error.
 =IFNA(formula, value_if_na) — catches only #N/A.
 =ISERROR(value) — returns TRUE if value is an error.

2.12 Pivot Tables & Charts

2.12.1 Pivot Tables

- **Insert:** Insert → Pivot Table.
- **Rows:** Categorical field to group by vertically.
- **Columns:** Categorical field to spread horizontally.
- **Values:** Numeric field to aggregate (SUM, COUNT, AVERAGE, etc.).
- **Filters:** Global filter for the whole pivot table.
- **Calculated Field:** Custom formula within the pivot (e.g., profit margin).
- Use **Slicers** (Insert → Slicer) for interactive filtering.

2.12.2 Charts

Chart Type	Best Used For
Bar / Column	Comparing categories
Line	Trends over time
Pie / Doughnut	Part-to-whole (use sparingly; max 5–6 slices)
Scatter	Correlation between two variables
Histogram	Distribution of a single variable
Area	Cumulative trend over time
Combo	Two different metrics on same chart
Sparkline	Tiny trend line inside a cell

2.13 Array Formulas

What is an Array Formula?

An array formula performs calculations on multiple values at once, returning a single result or a range of results. In older Sheets, wrap with `ARRAYFORMULA()`. Modern Sheets often spill automatically.

- `=ARRAYFORMULA(A1:A10 * B1:B10)` — multiplies element-wise.
- `=ARRAYFORMULA(SUM(IF(A1:A10="X", B1:B10, 0)))` — conditional sum without `SUMIF`.
- `=MMULT(array1, array2)` — matrix multiplication.

2.14 What-If Analysis

2.14.1 Goal Seek

Goal Seek

Used to find the input value needed to achieve a desired formula result.

Steps: Data → What-if Analysis → Goal Seek → Set cell, To value, By changing cell.

Example: What price gives \$10,000 profit?

2.14.2 Scenario Manager

Scenario Manager

Saves and compares multiple sets of input values (*scenarios*).

Steps: Data → What-if Analysis → Scenario Manager → Add scenarios → Show Summary.

Example: Best-case, worst-case, and base-case revenue projections.

2.15 Dashboard & Report Creation

- **Planning:** Define KPIs and audience before building.
- **Layout:** Use frozen rows/columns; group related charts spatially.
- **Slicers:** Insert → Slicer; link to pivot tables for interactive filtering.
- **Dynamic Ranges:** Use named ranges or `OFFSET` for auto-expanding data.
- **Protect Sheets:** Data → Protect Sheets and Ranges to prevent accidental edits.
- **Best Practices:** Consistent colours, clear labels, minimal clutter, include data source.

3 Python for Data Analysis

3.1 Key Libraries Overview

Library	Purpose
NumPy	Fast numerical arrays, linear algebra, random numbers
Pandas	Tabular data (DataFrames), data cleaning, analysis
Matplotlib	Low-level plotting and customisation
Seaborn	Statistical visualisations built on Matplotlib

Listing 1: Standard imports

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns

```

3.2 NumPy

3.2.1 NumPy Arrays

Listing 2: Creating arrays

```

1 a = np.array([1, 2, 3])           # 1D array
2 b = np.array([[1,2],[3,4]])      # 2D array
3 c = np.zeros((3, 4))            # 3x4 array of zeros
4 d = np.ones((2, 3))             # 2x3 array of ones
5 e = np.arange(0, 10, 2)         # [0, 2, 4, 6, 8]
6 f = np.linspace(0, 1, 5)        # 5 evenly spaced from 0 to 1
7 g = np.random.rand(3, 3)        # 3x3 random uniform [0,1)

```

Important Attributes

- `arr.shape` — tuple of dimensions, e.g. (3,4)
- `arr.ndim` — number of dimensions
- `arr.dtype` — data type (int64, float64, etc.)
- `arr.size` — total number of elements

3.2.2 Indexing & Slicing

Listing 3: Indexing and slicing

```

1 a = np.array([10, 20, 30, 40, 50])
2 a[0]           # 10 (first element)
3 a[-1]          # 50 (last element)
4 a[1:4]         # [20, 30, 40] (slice)
5 a[:2]          # [10, 20] (every 1st)
6
7 b = np.array([[1,2,3],[4,5,6],[7,8,9]])
8 b[1, 2]        # 6 (row 1, col 2)
9 b[:, 1]        # [2, 5, 8] (all rows, col 1)
10 b[0:2, 1:]     # [[2,3],[5,6]]

```

3.2.3 Boolean Masking

Listing 4: Boolean masking

```

1 a = np.array([5, 15, 25, 35])
2 mask = a > 10                # [False, True, True, True]
3 a[mask]                      # [15, 25, 35]
4 a[a % 2 == 0]                # Elements divisible by 2
5 a[(a > 10) & (a < 30)]      # Combine conditions with &

```

3.2.4 Vector, Numerical & Matrix Operations

Listing 5: NumPy operations

```

1 a = np.array([1, 2, 3])
2 b = np.array([4, 5, 6])
3
4 # Vector (element-wise) operations
5 a + b                # [5, 7, 9]
6 a * b                # [4, 10, 18]
7 a ** 2               # [1, 4, 9]
8
9 # Numerical / aggregate
10 np.sum(a)            # 6
11 np.mean(a)           # 2.0
12 np.std(a)            # 0.816
13 np.min(a)            # 1
14 np.cumsum(a)         # [1, 3, 6]
15
16 # Matrix operations
17 A = np.array([[1,2],[3,4]])
18 B = np.array([[5,6],[7,8]])
19 A @ B                # Matrix multiply (dot product)
20 np.dot(A, B)         # Same as A @ B
21 np.linalg.inv(A)     # Inverse of A
22 np.linalg.det(A)     # Determinant
23 A.T                  # Transpose

```

3.3 Pandas

3.3.1 Data Loading

Listing 6: Loading data

```

1 df = pd.read_csv("data.csv")
2 df = pd.read_csv("data.csv", index_col=0, parse_dates=['date'])
3 df = pd.read_excel("data.xlsx", sheet_name="Sheet1")
4 df = pd.read_json("data.json")
5
6 # Quick inspection
7 df.head(5)           # First 5 rows
8 df.tail(5)           # Last 5 rows
9 df.shape             # (rows, cols)
10 df.info()            # Column types and non-null counts
11 df.describe()        # Statistical summary of numeric columns
12 df.columns           # Column names
13 df.dtypes            # Data types per column

```

3.3.2 Data Analysing & Cleaning

Listing 7: Cleaning operations

```

1  # --- Missing values ---
2  df.isnull().sum()          # Count NaN per column
3  df.dropna()               # Drop rows with any NaN
4  df.dropna(subset=['age'])  # Drop only if 'age' is NaN
5  df.fillna(0)              # Replace NaN with 0
6  df['age'].fillna(df['age'].mean(), inplace=True) # Fill with mean
7
8  # --- Duplicates ---
9  df.duplicated().sum()     # Count duplicates
10 df.drop_duplicates()      # Remove duplicate rows
11 df.drop_duplicates(subset=['id']) # Based on specific column
12
13 # --- Data types ---
14 df['price'] = df['price'].astype(float)
15 df['date'] = pd.to_datetime(df['date'])
16 df['cat'] = df['cat'].astype('category')
17
18 # --- Inconsistent text ---
19 df['city'] = df['city'].str.strip().str.title()
20 df['city'].replace({'NY': 'New York'}, inplace=True)

```

3.3.3 Data Filtering

Listing 8: Filtering DataFrames

```

1  # Boolean filtering
2  df[df['age'] > 25]
3  df[(df['age'] > 18) & (df['city'] == 'Delhi')]
4  df[df['name'].str.contains('Kumar', na=False)]
5
6  # .loc and .iloc
7  df.loc[0:5, 'name':'age'] # label-based slice
8  df.iloc[0:5, 0:3]         # position-based slice
9  df.loc[df['status'] == 'Active', 'salary'] # Filter + select column
10
11 # query method (readable)
12 df.query("age > 25 and city == 'Mumbai'")

```

3.3.4 Data Grouping

Listing 9: GroupBy operations

```

1  # Single groupby
2  df.groupby('dept')['salary'].mean()
3  df.groupby('dept')['salary'].agg(['mean', 'sum', 'count'])
4
5  # Multiple keys
6  df.groupby(['dept', 'gender'])['salary'].mean()
7
8  # Custom aggregation
9  df.groupby('dept').agg(
10     avg_salary=('salary', 'mean'),
11     total_emp=('id', 'count'),

```

```

12 max_exp=('experience', 'max')
13 )

```

3.3.5 Data Merging

Listing 10: Merging DataFrames

```

1 # pd.merge (like SQL JOIN)
2 pd.merge(df1, df2, on='id', how='inner')    # Inner join
3 pd.merge(df1, df2, on='id', how='left')     # Left join
4 pd.merge(df1, df2, on='id', how='right')    # Right join
5 pd.merge(df1, df2, on='id', how='outer')    # Full outer join
6 pd.merge(df1, df2, left_on='emp_id', right_on='id') # Different key
   names
7
8 # pd.concat (stack rows or columns)
9 pd.concat([df1, df2], axis=0, ignore_index=True) # Stack rows
10 pd.concat([df1, df2], axis=1)                  # Stack columns

```

3.3.6 Data Pivoting

Listing 11: Pivot tables in Pandas

```

1 # pivot_table
2 pd.pivot_table(df, values='sales', index='region',
3                columns='product', aggfunc='sum', fill_value=0)
4
5 # pivot (no aggregation, unique index required)
6 df.pivot(index='date', columns='product', values='sales')
7
8 # melt (unpivot wide to long)
9 pd.melt(df, id_vars=['id', 'name'], value_vars=['q1', 'q2', 'q3'],
10         var_name='quarter', value_name='revenue')

```

3.4 Descriptive Statistics

Statistic	Formula / Notes
Mean	$\bar{x} = \frac{\sum x_i}{n}$ — sensitive to outliers
Median	Middle value; robust to outliers
Mode	Most frequent value
Variance	$\sigma^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$ (sample)
Std Dev	$\sigma = \sqrt{\text{Variance}}$
Range	Max — Min
IQR	$Q_3 - Q_1$; robust spread measure
Skewness	Asymmetry of distribution
Kurtosis	Tailedness of distribution

Listing 12: Descriptive stats in Python

```

1 df['col'].describe()          # count, mean, std, min, quartiles, max
2 df['col'].mean()
3 df['col'].median()
4 df['col'].mode()[0]

```

```

5 df['col'].var()
6 df['col'].std()
7 df['col'].skew()
8 df['col'].kurt()
9 df['col'].quantile([0.25, 0.5, 0.75])

```

3.5 Frequency Analysis

Listing 13: Frequency analysis

```

1 # Value counts
2 df['category'].value_counts() # Absolute frequency
3 df['category'].value_counts(normalize=True) # Relative frequency (
  proportion)
4
5 # Cross-tabulation
6 pd.crosstab(df['gender'], df['dept'])
7 pd.crosstab(df['gender'], df['dept'], normalize='index') # Row
  proportions

```

3.6 Correlation Analysis

Correlation

Measures the strength and direction of the linear relationship between two variables.

Pearson's r: ranges from -1 (perfect negative) to $+1$ (perfect positive). $r = 0$ means no linear relationship.

Listing 14: Correlation in Python

```

1 df.corr() # Pairwise correlation
  matrix
2 df['sales'].corr(df['price']) # Single pair
3
4 # Heatmap
5 sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
6 plt.show()

```

Interpretation

$|r| < 0.3$ = weak; $0.3 \leq |r| < 0.7$ = moderate; $|r| \geq 0.7$ = strong.

Correlation $\neq$ Causation!

3.7 Outlier Detection

3.7.1 IQR Method

Listing 15: Outlier detection with IQR

```

1 Q1 = df['col'].quantile(0.25)
2 Q3 = df['col'].quantile(0.75)
3 IQR = Q3 - Q1
4 lower = Q1 - 1.5 * IQR
5 upper = Q3 + 1.5 * IQR
6 outliers = df[(df['col'] < lower) | (df['col'] > upper)]

```

3.7.2 Z-Score Method

Listing 16: Outlier detection with Z-score

```
1 from scipy import stats
2 z_scores = np.abs(stats.zscore(df['col'].dropna()))
3 outliers = df[z_scores > 3]      # |z| > 3 is typical threshold
```

Use `sns.boxplot()` to visually inspect outliers. Box = IQR; whiskers = $1.5 \times \text{IQR}$; dots beyond whiskers = outliers.

3.8 Window Functions

Listing 17: Rolling and expanding windows

```
1 df['rolling_avg'] = df['sales'].rolling(window=7).mean()      # 7-day
   rolling mean
2 df['rolling_sum'] = df['sales'].rolling(window=3).sum()
3 df['cumulative'] = df['sales'].expanding().sum()              #
   Cumulative sum
4 df['pct_change'] = df['sales'].pct_change()                   # Period-
   over-period % change
5 df['rank'] = df['sales'].rank(method='dense', ascending=False)
```

3.9 Data Distributions

Distribution	Key Properties
Normal	Symmetric bell curve; mean = median = mode
Skewed Right	Long tail on right; mean > median
Skewed Left	Long tail on left; mean < median
Uniform	Every value equally likely
Bimodal	Two peaks; suggests two subgroups

Listing 18: Visualising distributions

```
1 sns.histplot(df['col'], kde=True)      # Histogram + KDE curve
2 sns.boxplot(x=df['col'])               # Box plot
3 sns.violinplot(x='category', y='val', data=df) # Violin (box +
   distribution)
```

3.10 Univariate, Bivariate & Multivariate Analysis

3.10.1 Univariate — One Variable

Listing 19: Univariate analysis

```
1 # Numeric
2 sns.histplot(df['age'], bins=20, kde=True)
3 sns.boxplot(x=df['salary'])
4
5 # Categorical
6 sns.countplot(x='dept', data=df)
7 df['dept'].value_counts().plot(kind='bar')
```

3.10.2 Bivariate — Two Variables

Listing 20: Bivariate analysis

```
1 # Numeric vs Numeric
2 sns.scatterplot(x='experience', y='salary', data=df)
3 sns.regplot(x='experience', y='salary', data=df) # With regression
   line
4
5 # Numeric vs Categorical
6 sns.boxplot(x='dept', y='salary', data=df)
7 sns.violinplot(x='gender', y='salary', data=df)
8
9 # Categorical vs Categorical
10 pd.crosstab(df['dept'], df['gender']).plot(kind='bar', stacked=True)
```

3.10.3 Multivariate — Three or More Variables

Listing 21: Multivariate analysis

```
1 # Pair plot (all pairwise combinations)
2 sns.pairplot(df[['age', 'salary', 'experience']], hue='dept')
3
4 # Heatmap of correlations
5 sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
6
7 # Scatter with colour and size encoding
8 plt.scatter(df['exp'], df['salary'], c=df['dept_code'],
9             s=df['age']*2, alpha=0.6, cmap='viridis')
```

4 Tableau

4.1 Interface

- **Data Pane** (left): Lists all fields — Dimensions on top, Measures below.
- **Shelves**: Rows, Columns, Pages — drag fields here to build views.
- **Marks Card**: Controls colour, size, label, detail, tooltip, shape.
- **Canvas/View**: The visualisation area.
- **Show Me**: Suggests chart types based on selected fields.
- **Sheet Tabs**: Create worksheets, dashboards, stories.

4.2 Data Types

Data Type	Icon / Notes
Number (Whole)	Integer
Number (Decimal)	Float
Date	Calendar
Date & Time	Calendar + clock
String	Abc
Boolean	T/F
Geographic (lat/lon)	Globe

4.3 Data Connections

Live vs Extract

- **Live**: Direct real-time connection to the source. Always current but slower for large data.
- **Extract (.hyper)**: Snapshot saved locally. Faster performance; scheduled refresh for updates.

4.4 Dimensions & Measures

Dimensions vs Measures

- **Dimensions**: Qualitative/categorical fields (e.g., Region, Product Name). Used to *slice* data.
- **Measures**: Quantitative fields (e.g., Sales, Profit). Aggregated by default (SUM, AVG, etc.).

Important

You can convert between Dimension and Measure. Discrete measures placed on Rows/Columns create headers; continuous measures create axes.

4.5 Discrete vs Continuous (Green vs Blue)

	Discrete (Blue)	Continuous (Green)
Values	Finite, individual	Infinite range
On shelf	Creates headers/labels	Creates axis
Example	Region, Month names	Sales, Date as range
Colour	Blue pill	Green pill

4.6 Marks Card & Charts

The Marks Card controls how data is visually encoded. Options: **Automatic, Bar, Line, Area, Square, Circle, Shape, Text, Map, Pie, Gantt, Polygon.**

Marks Property	Effect
Color	Encode a field as colour
Size	Encode a field as size of marks
Label	Show values on the marks
Detail	Disaggregate without visual encoding
Tooltip	Info shown on hover
Shape	Encode a field as different shapes

4.7 Filters & Their Types

1. **Extract Filter:** Applied to the data extract; reduces data at source.
2. **Data Source Filter:** Applied to the connection; affects all sheets using the source.
3. **Context Filter:** Creates a temporary table; other filters run against it. Improves performance. Set by right-clicking a filter → Add to Context.
4. **Dimension Filter:** Filters on categorical fields (include/exclude values).
5. **Measure Filter:** Filters on aggregated numeric values (e.g., SUM(Sales) > 1000).
6. **Table Calculation Filter:** Filters after table calculations are computed.

Filter Order of Operations

Extract → Data Source → Context → Sets → Dimension → Measure → Table Calc

4.8 Relationships, Joins, Union, Data Blending

Four Ways to Combine Data

Method	Description
Relationship	Tableau default (since 2020.2). Smart join deferred to query time. Preserves granularity. Recommended.
Join	SQL-style join at the row level (Inner, Left, Right, Full Outer). Fixed schema.
Union	Stacks tables with the same structure vertically (like SQL UNION ALL).
Data Blending	Joins data from <i>different</i> sources. Primary source + secondary. Uses aggregate-level link.

Use **Relationships** by default. Use **Joins** for single-source detailed control. Use **Blending** across different data sources. Use **Union** to stack sheets/files.

4.9 Calculated Fields & Table Calculations

4.9.1 Calculated Fields

- Created in the data pane (Analysis → Create Calculated Field or right-click).
- Evaluated at the **row level** (before aggregation) or on aggregates.
- Examples:
 - Profit Ratio: `SUM([Profit]) / SUM([Sales])`
 - Category check: `IF [Sales] > 1000 THEN "High" ELSE "Low" END`
 - Date part: `DATEPART('year', [Order Date])`

4.9.2 Table Calculations

Table Calculations

Applied **after** the query result is returned; work on values *in the view*.

Common types: `RUNNING_SUM`, `WINDOW_AVG`, `RANK`, `PERCENTILE`, `LOOKUP`, `FIRST`, `LAST`, `INDEX`.

Scope and direction: *Table, Pane, Cell; Down, Across, Down then Across*.

Listing 22: Table calculation examples (Tableau syntax)

```

1 RUNNING_SUM(SUM([Sales]))           // Cumulative sum
2 WINDOW_AVG(SUM([Sales]))           // Average over window
3 RANK(SUM([Sales]))                 // Rank in view
4 (SUM([Sales]) - LOOKUP(SUM([Sales]), -1)) / ABS(LOOKUP(SUM([Sales]), -1))
5                                     // Period-over-period growth

```

4.10 Level of Detail (LOD) Expressions

LOD Expressions

LOD expressions control the level of aggregation **independent** of the view's level of detail.

Syntax: `{FIXED/INCLUDE/EXCLUDE [dim1, dim2, ...] : AGG([measure])}`

4.10.1 FIXED LOD

```
{FIXED [Customer ID] : SUM([Sales])}
```

Computes the aggregate at the specified dimension(s), regardless of what is in the view. Ignores view-level filters (but respects context and data source filters).

Use case: Customer lifetime value, first order date per customer.

4.10.2 INCLUDE LOD

```
{INCLUDE [Product] : SUM([Sales])}
```

Adds the specified dimension(s) to the existing view level of detail. Result is then re-aggregated in the view.

Use case: Average sales per product per region (when only region is in view).

4.10.3 EXCLUDE LOD

```
{EXCLUDE [Month] : SUM([Sales])}
```

Removes the specified dimension(s) from the view's level of detail.

Use case: Year-to-date vs total — compare monthly sales to annual total in the same view.

LOD Type	Granularity	Filter Interaction
FIXED	Specified dims only	Ignores dimension filters; respects context
INCLUDE	View dims + specified	Respects all filters
EXCLUDE	View dims - specified	Respects all filters

4.11 Sets & Parameters

4.11.1 Sets

Sets

A **Set** is a custom field that defines a subset of data based on a condition.

- **Fixed Set:** Manually selected members. Static.
- **Dynamic Set:** Condition-based (e.g., top 10 customers by sales). Updates with data.
- Sets appear in the data pane under Dimensions.
- **In/Out:** A field showing whether each record is inside or outside the set.
- **Set Actions:** Allow dashboard interactions to update set membership.
- **Combined Sets:** Union, intersect, or difference of two sets.

4.11.2 Parameters

Parameters

A **Parameter** is a dynamic, user-controlled input variable that can replace a constant in calculations, filters, or reference lines.

- Types: Integer, Float, String, Boolean, Date.
- Input methods: Free type, List, Range with step.

- **Usage:** Swap measures dynamically, set top-N thresholds, adjust bin sizes.
- **Example:** CASE [Select Metric] WHEN "Sales" THEN [Sales] WHEN "Profit" THEN [Profit] END

4.12 Groups, Bins, Hierarchies

4.12.1 Groups

- Manually combine dimension members into higher-level categories.
- Right-click on members in view → Group; or right-click field → Create → Group.
- A new grouped field is created in the Data Pane.

4.12.2 Bins

- Group continuous numeric measures into discrete buckets (like a histogram).
- Right-click measure → Create → Bins → specify bin size.
- Use the bin field as a Dimension to create histograms.
- Bin size can be controlled by a Parameter for dynamic binning.

4.12.3 Hierarchies

- Allow drill-down in views (e.g., Year → Quarter → Month → Day).
- Create by dragging one dimension onto another in the Data Pane.
- Built-in date hierarchy: Year, Quarter, Month, Week, Day.
- +/- icons on axes let users expand/collapse hierarchy levels.

4.13 Containers, Tiles (Layout Containers)

Layout Containers

Used in **Dashboards** to organise and align objects automatically.

- **Horizontal Container:** Objects arranged side-by-side; auto-resizes width.
- **Vertical Container:** Objects stacked top-to-bottom; auto-resizes height.
- **Tiled vs Floating:** Tiled objects snap to the grid; Floating objects can be placed freely.
- Nest containers inside each other for complex layouts.
- Use the **Layout Pane** to fine-tune padding, borders, and sizes.

Use containers (not free float) for consistent, responsive layouts. A common pattern: outer Vertical container → Header row + Content horizontal container + Footer row.

4.14 Dashboard Design & Storyboard

4.14.1 Dashboard Design

- **Size:** Fixed, Automatic, or Range. Fixed = pixel-perfect; Automatic = responsive.
- **Add Sheets:** Drag worksheets from the left panel.
- **Add Objects:** Text, Images, Web Page, Blank, Navigation, Extension.
- **Actions:** Add interactivity — Filter Action, Highlight Action, URL Action, Set Action, Parameter Action.

- **Filter Action:** Clicking a mark filters other sheets.
- **Highlight Action:** Clicking a mark highlights related marks.
- **URL Action:** Opens a URL based on a field value.
- **Tooltips:** Customise with field values; can embed viz-in-tooltip.
- **Design principles:** Clear title, consistent colour scheme, guide the viewer's eye top-to-bottom, left-to-right.

4.14.2 Story (Storyboard)

Tableau Story

A **Story** is a sequence of worksheets and dashboards arranged in **story points** to convey a narrative.

- Each story point = a snapshot of a view with a caption.
- Navigate with arrows or a dot/tab navigator.
- Story points can **blank out** (new layout) or **duplicate** the previous point.
- Use to present findings in a logical flow: context → insight → recommendation.

Exam Preparation Summary

Top Topics to Revise

1. **VLOOKUP / XLOOKUP / INDEX-MATCH** — know the syntax and differences cold.
2. **SUMIFS / COUNTIFS / AVERAGEIFS** — argument order and wildcard usage.
3. **Pandas merge, groupby, pivot_table** — most common interview/exam topics.
4. **Outlier Detection** — IQR formula: $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$.
5. **LOD Expressions in Tableau** — FIXED vs INCLUDE vs EXCLUDE with examples.
6. **Filter Order in Tableau** — Extract → Data Source → Context → Dimension → Measure → Table Calc.
7. **Boolean Masking & Array Slicing in NumPy**.
8. **Table Calculations** — RUNNING_SUM, RANK, LOOKUP.
9. **Types of Analytics** — Descriptive / Diagnostic / Predictive / Prescriptive.
10. **Correlation** — Pearson r interpretation and heatmap using Seaborn.

Common Mistakes to Avoid

- Confusing SUMIF argument order with SUMIFS (sum range position).
- Using TRUE in VLOOKUP when you need exact match — always use FALSE / 0.
- Forgetting inplace=True in Pandas or not reassigning the result.
- Mixing up .loc (label-based) and .iloc (integer position-based).
- Applying FIXED LOD and expecting it to respect dimension filters — it does not.
- Placing a continuous field on Rows/Columns expecting a header — it creates an axis.